

```
%10/02/2012
%Piter Garcia
%Homework Number 3
```

```
box = imread('Box.jpg');
ML = imread('MauiLab.jpg');
PO = imread('PatternOne.jpg');
OF =
imread('OrangeFlower.jpg');
```

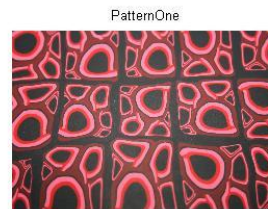
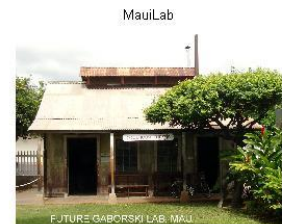
```
screen_size = get(0,
'ScreenSize');
```

```
f1 = figure(1);
set(f1, 'Position', [0 0
screen_size(4) screen_size(4) ]
);
subplot(2, 2, 1), imshow(box),
title('Box');
subplot(2, 2, 2), imshow(ML),
title('MauiLab');
subplot(2, 2, 3), imshow(PO),
title('PatternOne');
subplot(2, 2, 4), imshow(OF),
title('OrangeFlower');
```

```
for x=1:4,

    if x==1
        Jg = rgb2gray(box);
    end;

    if x==2
        Jg = rgb2gray(ML);
    end;
    if x==3
        Jg = rgb2gray(PO);
    end;
    if x==4
        Jg = rgb2gray(OF);
    end;
```



In this section I am reading the four images from the matlab folder and assigning to the shown variables above. Then, I display them so we know the images we will be working with. The for loop and if statements shown after the images have been included to minimize code. What this means is that I will be using the same code to find the edges and to apply different edge tests and sigma to four images with the code one image.

```
Jg = im2double(Jg);
Jg = imresize(Jg, .25);
figure, imshow(Jg);title('');

f = fspecial('sobel');
imEdge = imfilter(Jg, f );
figure, imshow(imEdge,[]);title('fspecial
and filtered - Horizontal Edge');
```

```
max(imEdge(:))
min(imEdge(:))
```

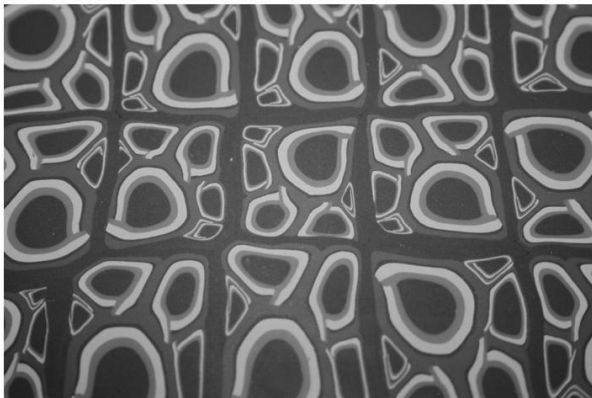
Resized and Rgb2Gray



Resized and Rgb2Gray



Resized and Rgb2Gray

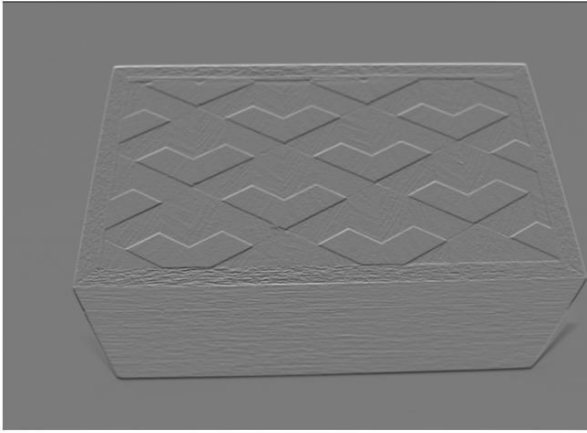


Resized and Rgb2Gray

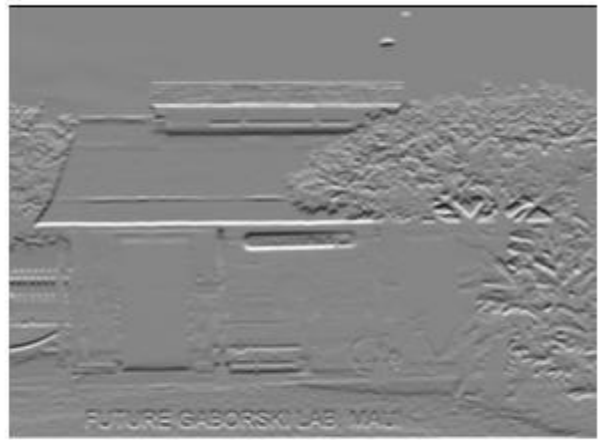


In this section, by using for loops along with if statements explained above. I am able to display all for images, one at a time, after having gone through the process of being doubled, resized, and gray-scaled. Afterwards, I proceed to find the horizontal edge image of each one. However, those images are shown and explain below, followed by the display of the maximum and minimum range of the horizontal edge image.

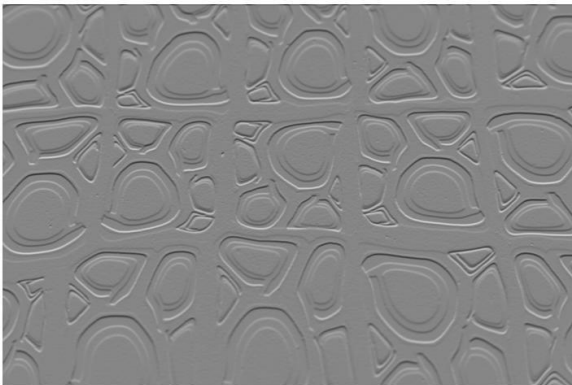
fspecial and filtered - Horizontal Edge



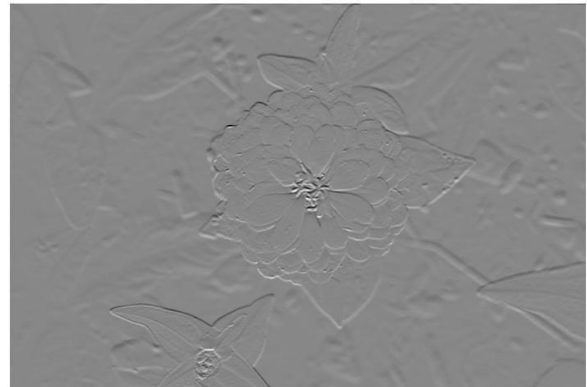
fspecial and filtered - Horizontal Edge



fspecial and filtered - Horizontal Edge



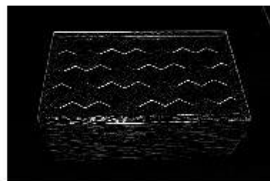
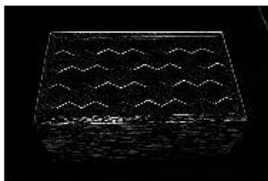
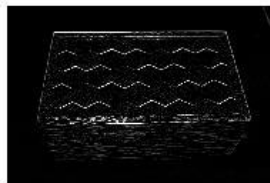
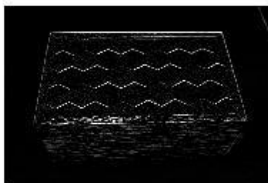
fspecial and filtered - Horizontal Edge



This is a follow up from the above method. Here, I am able to display all for images, one at a time, after the above processed and also after applying the sobel and filter function to find the horizontal edge image of each one, the threshold value used is the default.

```
figure,
for i=1:4,
    subplot(2, 2, i),imshow(imEdge,i);
end;
suptitle('Thresholding Horizontal Edges with Sigma from 1-4');
```

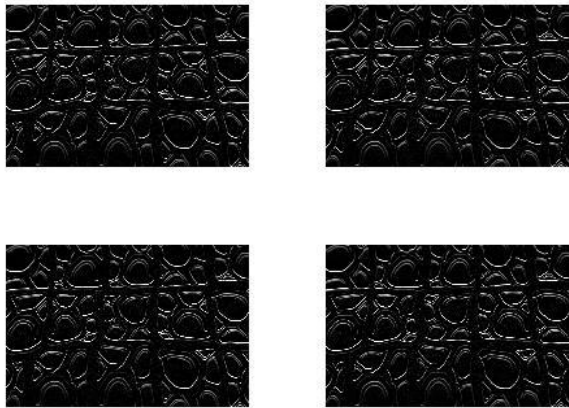
Thresholding Horizontal Edges with Sigma from 1-4



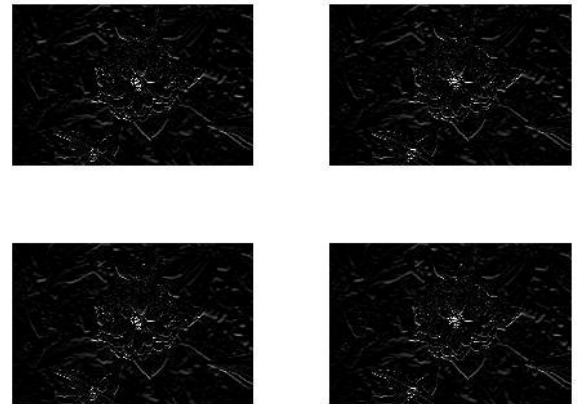
Thresholding Horizontal Edges with Sigma from 1-4



Thresholding Horizontal Edges with Sigma from 1-4

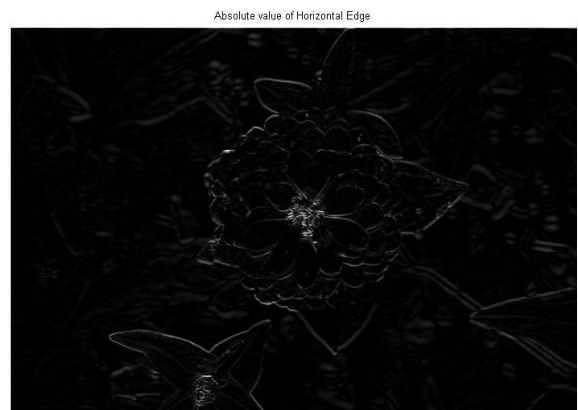
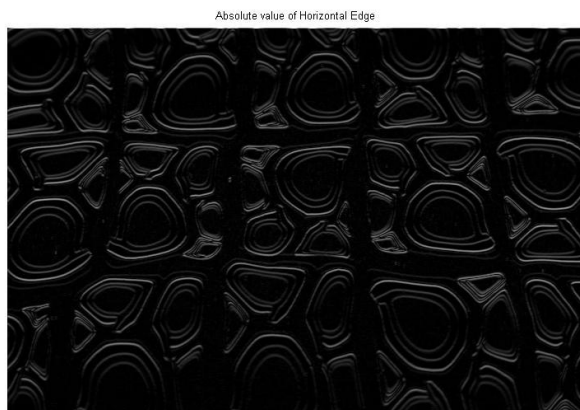
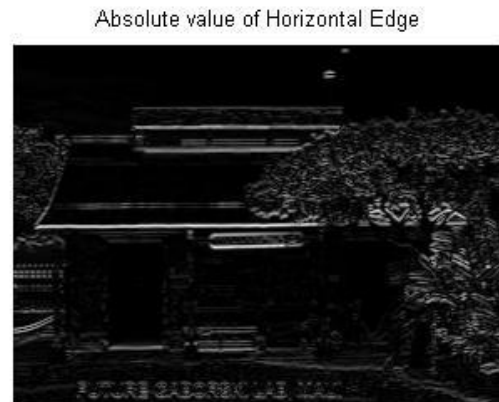
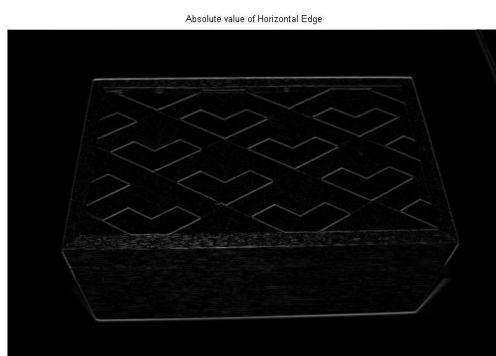


Thresholding Horizontal Edges with Sigma from 1-4



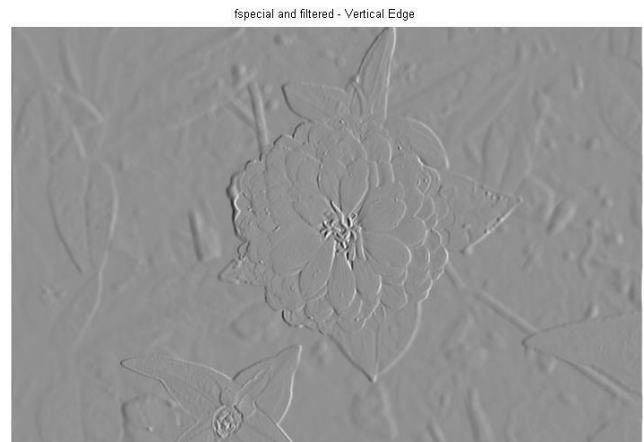
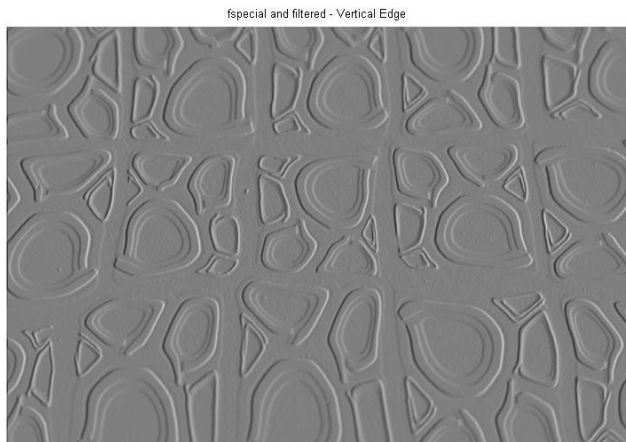
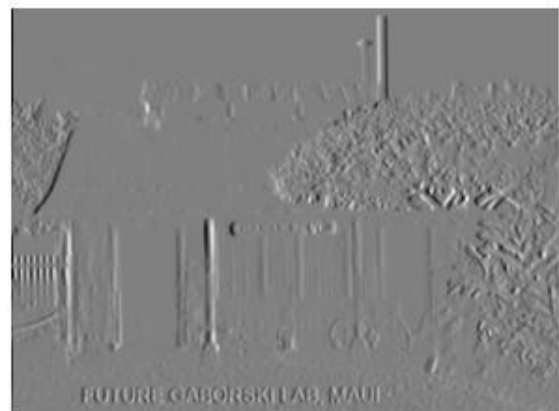
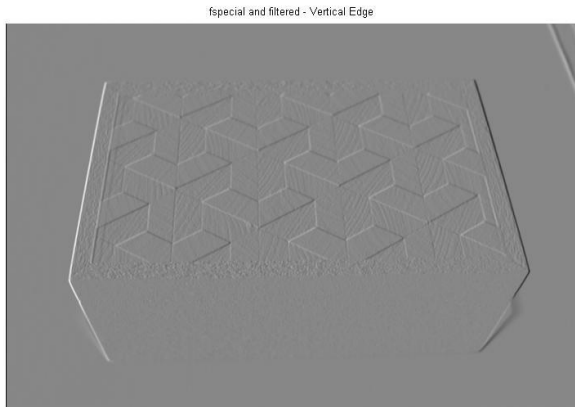
In this section I used the same function as the one explained above. The only difference is that I used different values for sigma, from 1 to 4. As we can observe and compare, this worked well for the first and third image but not for the second the forth image. I believe that, this kind of process is only good for images that do not have a lot of details in them, meaning images of only on object or objects whose reflection does not produce as much noise as the one observed in the second image. Therefore, this method works but not in every case.

```
figure, imshow(abs(imEdge),[]);title('Absolute value of Horizontal Edge');
```



Since a lot of noise was being displayed after the horizontal edge image process. I proceeded to get the absolute of the main horizontal edge image processed, and the output was successful as we can tell from the above images. Most of the noise disappeared and now the images, the four of them, can be more visible or, to be more specific, their edges are more noticeable.

```
fv=f';
imEdgeV = imfilter(Jg, fv );
figure, imshow(imEdgeV,[]);title('fspecial and filtered - Vertical Edge');
```

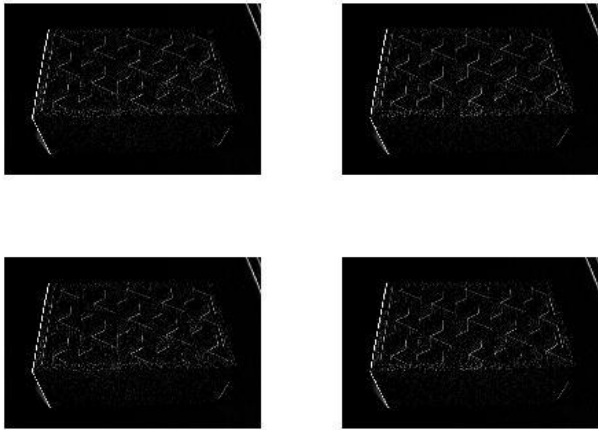


In this section, as it was mentioned at first, I am using for loops along with if statements explained above. I am able to display all for images, one at a time, after having gone through the process of being doubled, resized, and gray-scaled. Afterwards, I have proceeded to find the vertical edge image of each one. However, those images are shown and explain below, followed by the display of the maximum and minimum range of the horizontal edge image.

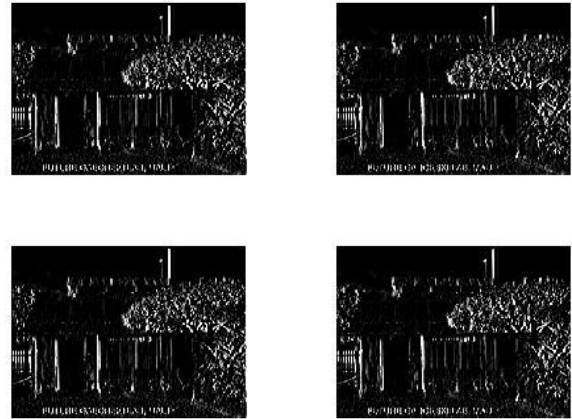
```
figure,
for i=1:4,
    subplot(2, 2, i),imshow(imEdgeV,5);
end;
suptitle('Thresholding Vertical Edges with Sigma from 1-4');

figure, imshow(abs(imEdgeV),[]);title('Absolute value of Vertical Edge');
max(fv(:));
min(fv(:));
```

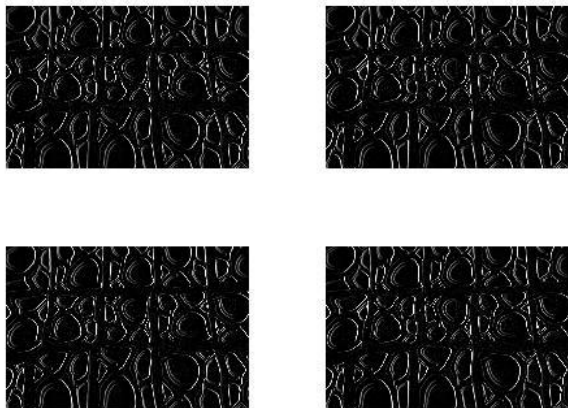
Thresholding Vertical Edges with Sigma from 1-4



Thresholding Vertical Edges with Sigma from 1-4



Thresholding Vertical Edges with Sigma from 1-4



Thresholding Vertical Edges with Sigma from 1-4



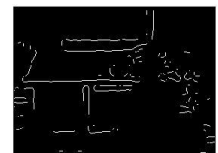
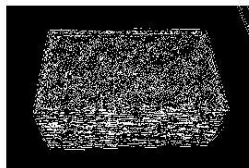
In this section I used the same function as the one explained above. The only difference is that I used different values for sigma, from 1 to 4. As we can observe and compare, this worked well for the second and third image but not for the first and forth image. I believe that, this kind of process is only good for images that do not have a lot of details in them, meaning images of only on object or objects whose reflection does not produce as much noise as the one observed in the second image. Therefore, this method works but not in every case.

```

figure,
for i=1:4,
    t=edge(Jg,'log',[ ],i);
    subplot(2, 2, i),imshow(t);
end;
suptitle(' Laplacian of Gaussian filtered Image');

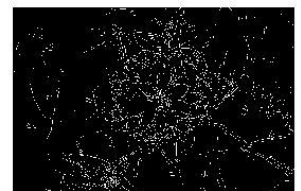
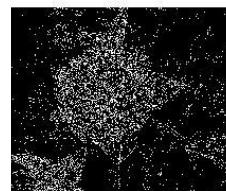
```

Laplacian of Gaussian filtered Image

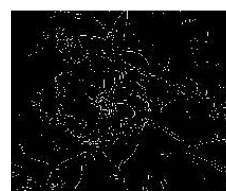


Laplacian of Gaussian filtered Image

Laplacian of Gaussian filtered Image



Laplacian of Gaussian filtered Image



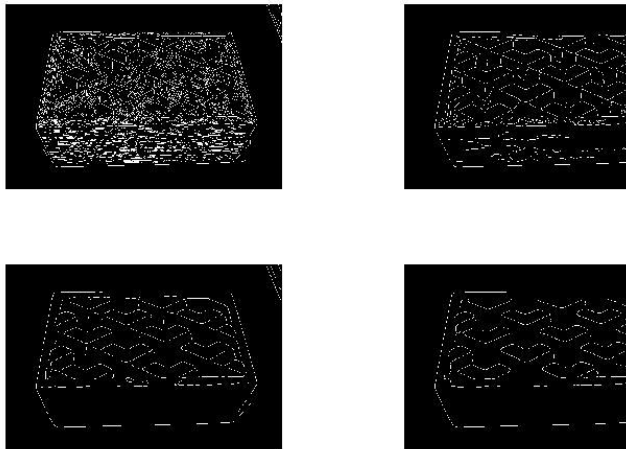
Here I proceeded to implement the Laplacian of Gaussian filtered to the gray-scale processed image. The results, as we can observe, were successful since this method seems to have worked for every image used in this project. The laplacian of Gaussian filter, to my understanding, had a more successful output because its function is based on 20 or db. This means that this function is applied based on a logarithmic method, which takes care of most of the noise in the image without distorting the image.

```

figure,
for i=1:4,
    t=edge(Jg,'canny',[ ],i);
    subplot(2, 2, i),imshow(t);
end;
suptitle('Canny filtered Image');
end;

```

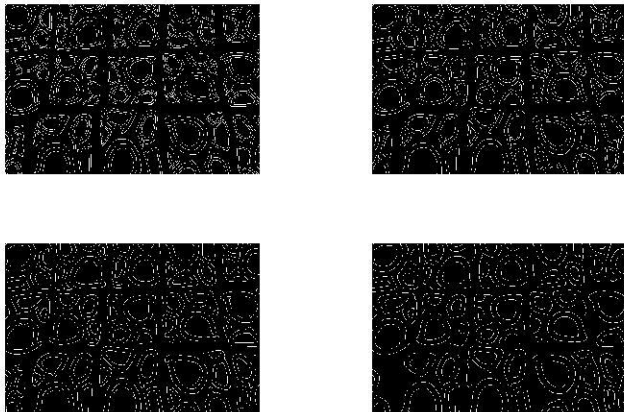
Canny filtered Image



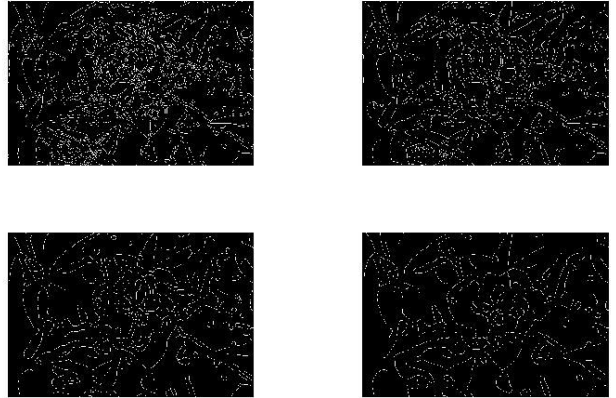
Canny filtered Image



Canny filtered Image



Canny filtered Image



This is canny process, a very successful method as we can see. For a point of view you can tell that the output obtained from this method was almost as good as the Laplacian. There reason why is because only some details seems to be more visible in this method than in the previous one. However, seeing from an overall point of view, all images edges are more clear in the previous method.

Write Up

For the construction of this project I first proceeded to load the pictures to Matlab and assigned each one to a respective variable which later was used to modify each picture in order to detect the edges of each with different method to compare and realize which out of all the methods applied worked better in terms or more visibility.

First:

For an easier construction, less coding, and faster way to implement, see and compare different method of image edging process I used a for loop that took each picture to be processed separately and implanted different edging methods which later were going to be compared.

Second:

Each image, according to the loop, was processed (read) and transformed to the desired input; transformed to a double, resized, and turned to a gray scale for a better and response from the image edging methods. Note: all original images were displayed to keep in mind the images we are going to be working with throughout the conversion process.

Third:

From each image was taken the horizontal edge image. Then, a for loop was used to obtain different outputs by changing the sigma of the horizontal edge image. Meaning the horizontal image was threshold to a value from 1-4. Details of the process are given the process explanation sheet given attached with this write up.

After the above step was done, I proceeded to implement the absolute value of the give horizontal edge image result. As a result, the image noise was less noticeable and the images edges were more clear since the absolute resulting truncates the pixel values them so that values smaller than 0 are made 0 and values larger than 255 are set to 255. For negative values, you can also take the absolute value instead.

Forth:

From each image was taken the vertical edge image. Then, a for loop was used to obtain different outputs by changing the sigma of the horizontal edge image. Meaning the vertical image was threshold to a value from 1-4. Details of the process are given the process explanation sheet given attached with this write up. After the above step was done, I proceeded to implement the absolute value of the give horizontal edge image result. As a result, the image noise was less noticeable and the images edges were more clear since the absolute resulting truncates the pixel values them so that values smaller than 0 are made 0 and values larger than 255 are set to 255. For negative values, you can also take the absolute value instead.

Fifth:

I implement the Laplacian of Gaussian filter to each image . The resulting image was successful, and the reason why is the following. Laplacian filter finds areas of rapid change (edges) in images. Since derivative filters are very sensitive to noise, it is common to smooth the image before applying the Laplacian. This is why it gives as a result such smooth image in this case.

Sixth:

I implant the Canny filter to each image to detect their edges as follow. This edge detection operator uses a multi-stage algorithm to detect a wide range of edges in images. This is why the Canny filter was also just as good as the Laplacian filter. Some details, that I appreciate, made me pick the previous result as a better result. Nevertheless, they both show a very good representation of image detection.

Overall, the operator response is strongly dependent on the relationship between the size of the structures in the image domain and the size of the Gaussian kernel used for pre-smoothing. In order to automatically capture t of different size in the image domain, a multi-scale approach is therefore necessary. This other approach is the Difference of Gaussians (DoG), this operator is in essence similar to the Laplacian and can be seen as an approximation of the Laplacian operator. In a similar fashion as for the Laplacian blob detector, blobs can be detected from scale-space extrema of differences of Gaussians.